

ATDRPS

Advanced Threat Detection and Reinforcement Protocol System

A plain-language explainer of what was built, how it works,
and answers to the questions judges are likely to ask

Event	Smart India Hackathon 2026
Problem Statement	26153 -- AI-based Network Attack Forecasting from Network Traffic Data
Organisation	National Technical Research Organisation (NTRO)
Team	Bell Labs -- Lloyd Institute of Engineering & Technology

This document assumes no coding or cybersecurity background. Every technical term is explained the first time it appears. Read Part 1 to be able to explain the project in your own words; use Part 4 (the Q&A) as cue cards during judging.

Contents

- Part 1 -- The One-Minute Explanation
- Part 2 -- How It Works, Step by Step (Backend)
- Part 3 -- What the Judge Sees On Screen (Frontend / Dashboard)
- Part 4 -- The Numbers (Results, in Plain Language)
- Part 5 -- What's Real vs What's a Bonus
- Part 6 -- Judge Q&A (organised by topic)
- Part 7 -- Cheat Sheet (memorise these)
- Part 8 -- Glossary (every term explained)

Part 1 -- The One-Minute Explanation

The problem, in one line

Normal intrusion detection tools (an "IDS", Intrusion Detection System -- software that watches network traffic for attacks) ask **"is this one conversation malicious, right now?"** They look at one flow (one conversation between two computers) at a time and forget everything that happened before or after it.

The problem is that a real attack is not one packet -- it is a **sequence** of steps over time (scan a bit, break in, move around, steal data). A slow, careful attacker stays under the radar of every single-flow check, because no single flow looks suspicious on its own.

What ATDRPS does differently

ATDRPS asks a different question: **"given everything that has happened on this network so far, where is it heading next?"** It watches the network in 30-second chunks, learns the normal "rhythm" of traffic, and then forecasts several steps into the future -- like a weather forecast, but for a cyber-attack instead of rain.

This kind of model -- one that learns how a system's state changes over time and can simulate it forward -- is called a **"world model"** in AI research. That is where the name of the technique comes from.

What it actually outputs

- A **probability** (0 to 100%) that the network is being infiltrated, for right now and for the next several time-windows ahead.
- The **predicted attack stage** using the industry-standard MITRE ATT&CK framework (explained in Part 8) -- e.g. "Reconnaissance" (scanning), "Lateral Movement" (spreading inside the network), "Exfiltration" (stealing data).
- A plain-English **explanation** of *why* the model thinks so -- e.g. "a lot of traffic is going to remote-access ports like SSH/RDP" -- so it is never a black box.
- The actual network **flows** (conversations) behind the alert, so an analyst can verify it themselves.

Why this matters for NTRO / critical infrastructure

The problem statement is from NTRO (National Technical Research Organisation), which deals with national security and often air-gapped networks (networks deliberately disconnected from the public internet, for safety). So ATDRPS is built to run **completely offline** -- no cloud calls, no internet needed, nothing ever leaves the machine it runs on.

Part 2 -- How It Works, Step by Step (Backend)

"Backend" means the part of the program that does the actual work behind the scenes -- reading files, doing the maths, running the AI model. It has no buttons or screens of its own; the dashboard (Part 3) is just a window into it.

Step 1 -- Input

The user gives it a **PCAP file** (a raw recording of network packets, the same format Wireshark uses) or a **flow CSV** (a spreadsheet-style summary of network conversations, the format tools like CICFlowMeter produce). This matches exactly what the official problem statement asks for: "a PCAP or CSV file as input".

Step 2 -- Parsing

ATDRPS reads the raw file itself using code written for this project -- it does **not** depend on external libraries like *scapy* or *pyshark*. This was a deliberate choice: fewer outside dependencies means fewer things that can break, and fewer places a security vulnerability could sneak in -- important on a network that isn't supposed to touch the internet.

Step 3 -- Flow assembly

Individual packets are grouped into **flows** -- a flow is one "conversation" between two computers, e.g. "laptop A talked to server B on port 443 (HTTPS) for 3 seconds and sent 40 packets". This is the same idea NetFlow/CICFlowMeter use.

Step 4 -- Feature extraction (dual-level)

For every flow, the system calculates measurable characteristics called **features** -- numbers that describe the traffic. It calculates two kinds at once:

- **Flow-level features** (also available from a CSV): duration, byte/packet counts, TCP flag patterns, timing between packets.
- **Packet-level features** (only available from a full PCAP, not a CSV): TTL (time-to-live, a small counter in every packet -- attackers often show inconsistent TTLs), TCP window size, retransmissions, out-of-order packets, fragmentation.

Using both is a real advantage this project makes explicit: a slow port scan (checking many ports one by one) can hide from flow-level stats alone, but its constant TTL and tiny fixed packet size give it away at the packet level.

Step 5 -- Time windowing

All the flows are grouped into **30-second windows**. For each window, ~100 features are summarised into one **state vector** (a list of 103 numbers describing "what the whole network looked like in this 30-second slice") -- counts and rates, statistical spread (mean/std/max) of flow features, and 21 hand-built "behavioural detectors" tuned to specific attack patterns (port-scan detector, C2-beacon-regularity detector, DNS-tunnelling detector, SYN-flood half-open-connection detector, and more).

Step 6 -- The world model (the core AI)

A sequence of these 30-second state vectors (the last 16 windows = 8 minutes of history) is fed into the model. The model has three jobs at once:

- Predict how the network **state will change** in the next window (this is what makes it a "world model" rather than a plain classifier -- it learns cause-and-effect over time, not just "attack / not attack").
- Predict the **MITRE ATT&CK stage** of the next window (6 categories, see Part 8).

- Predict the **infiltration probability** (a single 0-100% risk score) for the next window.

There are two versions of this model, explained fully in Part 5: a full **temporal transformer** (a modern deep-learning architecture, the same family used in large language models, just much smaller) and a lighter **linear-dynamics** version that needs no heavy AI libraries at all -- this second one is what actually ships and runs in the demo.

Step 7 -- K-step forward simulation

Once the model can predict "one window ahead", it is run **five times in a row**, each time feeding its own prediction back in as the new "current state" -- like predicting tomorrow's weather, then using that guess to predict the day after, and so on. This produces a probability **timeline** for the next five windows (2.5 minutes ahead), not just a single number. "K-step" simply means "K windows into the future"; here K = 5.

Step 8 -- Explainability

Every prediction comes with a breakdown of **which features caused it**, using a technique called SHAP (SHapley Additive exPlanations -- explained in Part 8), implemented from scratch for this project. It also shows **which past time-windows** mattered most to the prediction, and cross-checks the AI's answer against a separate set of hand-written, human-readable rules (e.g. "if more than 20 distinct ports were touched by one source, flag it as scanning") so a human can sanity-check the AI.

Step 9 -- Output

All of the above is packaged into one result: the risk timeline, the predicted stage, the 5-step forecast, the explanation, and the actual flows involved -- shown either in the terminal (command line) or in the dashboard (Part 3).

Part 3 -- What the Judge Sees On Screen (Frontend / Dashboard)

"Frontend" means the part the user actually sees and clicks on. ATDRPS's frontend is a single web page, served locally by a small Python web server called **Flask** -- there is no separate "app" to install, and no data is sent anywhere; the page and the analysis both run on the same machine, and it works with no internet connection.

What you see, top to bottom

- **Header** -- project name, and status badges confirming "fully offline", the window size (30s), the forecast horizon (K=5), and which model is active.
- **Upload box** -- drag-and-drop (or click to browse) a .pcap/.pcapng/.cap/.csv file, plus two settings: alert threshold (how high the risk score must go before it's flagged) and horizon K (how many windows ahead to forecast). One "Forecast" button runs everything.
- **Stat cards** -- four at-a-glance numbers: Peak Risk (%), Predicted Stage, Windows Analysed, and Flows in the Capture, plus a circular gauge showing the peak risk visually.
- **Risk-over-time chart** -- a line graph of the infiltration probability for every observed window, with a dashed line marking the alert threshold, and a hatched region on the right showing the K-step forecast into the future.
- **Forward simulation cards** -- one card per forecast step (+1 to +5), each showing the predicted stage and probability, with confidence naturally decaying the further out the forecast goes (an honest model admits it gets less sure the further ahead it looks).
- **"Why" panel** -- the SHAP explanation as a ranked bar chart of contributing features in plain English, plus which past time-windows mattered most.
- **Flow table** -- the real network conversations (source IP:port, destination IP:port, protocol, packet/byte counts, TCP flags) inside the highest-risk window, so a human analyst can verify the call themselves rather than trusting a black box.

Is the dashboard real, or a mockup?

It is real. The dashboard's "Forecast" button calls the exact same analysis engine as the command-line tool -- there is no fake or pre-recorded data behind it. This was verified today by uploading a real packet capture through the dashboard and confirming the numbers, the chart, and the flow table all matched a fresh, live run of the underlying engine.

Part 4 -- The Numbers (Results, in Plain Language)

All numbers below come from held-out test data -- traffic the model never saw during training, recorded from completely different simulated "campaigns" than the training data, which is the honest way to measure a model (see "data leakage" in the Q&A).

The headline comparison

Three models were trained and compared on the exact same data and the exact same features, so the comparison is fair:

Model	F1 score	False-positive rate	Stage accuracy
Logistic regression (static) -- the traditional "one flow at a time" IDS	0.838	16.6%	75.6%
Logistic regression (with context) -- a stronger baseline that also sees history	0.895	8.9%	80.1%
ATDRPS world model	0.921	7.1%	84.0%

F1 score: a single number (0 to 1) balancing how many real attacks were caught against how many false alarms were raised -- higher is better. False-positive rate: the percentage of normal traffic wrongly flagged as an attack -- lower is better, since too many false alarms make analysts ignore the tool. Stage accuracy: how often the predicted MITRE stage matched the true stage.

In plain words

- ATDRPS's world model beats a traditional single-flow classifier by roughly 10 F1 points, and **cuts false alarms by more than half** (7.1% vs 16.6%).
- It still beats the stronger "sees history but doesn't simulate forward" baseline -- proving the forecasting/simulation idea itself adds value, not just "looking at more data".
- The advantage **grows** the further into the future you forecast, because a model that only learned to classify a single moment has nothing to extrapolate from, while a model that learned real cause-and-effect dynamics can.

Does it actually understand cause-and-effect, or is it just guessing well?

This was checked directly. A "do-nothing" baseline ("assume nothing changes") gets a prediction error of 1.006. ATDRPS's model gets 0.611 -- meaningfully better than doing nothing, which is the test for whether a model has actually learned how the network state evolves, rather than just memorising patterns.

Where it's weakest (said honestly, not hidden)

The "Impact" stage (denial-of-service floods and ransomware encryption) scores only 0.50 F1, because the test set only had 32 examples of it -- simply too few to learn well. Every other stage scores between 0.78 and 0.90. Being upfront about this is intentional: claiming perfect scores everywhere is the fastest way to lose credibility with a technical judge.

Speed

On a real capture of 88,767 packets (about 4,737 conversations), the entire pipeline -- parsing, feature extraction, windowing, forecasting, and generating the explanation -- runs in **under 4 seconds** on an ordinary laptop, with no GPU required.

Part 5 -- What's Real vs What's a Bonus

It's worth being precise about this, because a judge who has read the official problem statement may probe exactly this line.

What the official problem statement actually asks for

PS 26153 asks for a tool that takes **a PCAP or CSV file as input** and forecasts attacks with MITRE stage mapping and explainability, compared against a baseline classifier. It does **not** ask for live network capture.

What is core, required, and fully built

- PCAP and flow-CSV file input, exactly as specified.
- The world-model forecasting engine (K-step rollout, MITRE stage prediction).
- SHAP + attention/occlusion explainability (mandatory requirement).
- The logistic-regression baseline comparison (mandatory requirement).
- The offline dashboard and CLI.

What is a bonus, built but not the centrepiece of the demo

A real, working **live packet capture mode** was also built -- it can sniff real network traffic in real time (on Linux or Windows, needs admin/root privileges to read raw packets) and run it through the exact same forecasting engine live. It is genuinely functional and was tested end-to-end with real traffic. However, since the official problem statement only asks for file input, this is kept as a bonus talking point ("we also built live capture, beyond what was asked") rather than something the core demo depends on -- so nothing about the graded deliverables is at risk if live capture behaves unpredictably on demo day (different network, different drivers, needing Administrator rights, etc.).

If a judge asks to see live capture, it is safe to demonstrate -- it works. Just don't lead with it, and don't imply the official grading depends on it.

Part 6 -- Judge Q&A

Organised by the angle a judge is likely to come from. Answers are written short and spoken, the way you'd actually say them out loud -- not textbook definitions.

A. "Why is this different / why does it matter" questions

Q. How is this different from a normal antivirus or IDS?

A. A normal IDS looks at one connection at a time and asks "is this bad, right now?". ATDRPS looks at the last 8 minutes of the whole network and asks "where is this heading?" -- it can catch attacks that look innocent flow-by-flow but form a clear pattern over time, like a slow port scan.

Q. What is a "world model"? Explain it like I'm not technical.

A. It's an AI that learns how a system changes step by step, so it can simulate the future instead of just labelling the present. Think of it like a weather model: it doesn't just say "it's raining right now", it learns the physics well enough to forecast tomorrow.

Q. Why forecast the future instead of just detecting the attack when it happens?

A. Because by the time an attack is fully visible, damage may already be done. A few minutes of early warning -- "this looks like it's heading toward data theft" -- gives a security team time to act before the worst stage happens.

Q. Why do you need MITRE ATT&CK stages at all? Why not just say "attack" or "no attack"?

A. MITRE ATT&CK is the industry-standard vocabulary security teams already use to describe attacker behaviour (scanning, breaking in, spreading, stealing data, etc.). Naming the stage tells an analyst what to do next, not just that something is wrong.

B. Technical / machine-learning questions

Q. What model architecture are you actually using?

A. The full design is a temporal transformer -- the same family of architecture used in modern language models, just far smaller, applied to sequences of network states instead of text. Because that needs PyTorch (a heavy deep-learning library) which may not always be installed, the project also has a lightweight "linear-dynamics" version (ridge regression + logistic regression) that implements the exact same world-model idea without any heavy dependencies -- and that's the version that actually ships and runs the demo.

Q. Isn't having two models redundant?

A. No -- the second, lightweight model does two jobs: it keeps the whole system runnable in a restricted, offline environment, and it acts as a fair scientific check: "does the fancier transformer actually earn its complexity, or does a well-built simple model do just as well?" Reporting that honestly is better than hiding the comparison.

Q. How is this actually better than a plain logistic regression classifier? Prove it.

A. Same features, same data, same split -- the world model gets F1 0.92 vs 0.84 for the static classifier, and less than half the false-alarm rate (7.1% vs 16.6%). The gap also widens the further ahead you forecast, which a snapshot classifier structurally cannot do.

Q. What is SHAP and why is explainability mandatory here?

A. SHAP is a mathematically grounded way to answer "which input features pushed this prediction up or down, and by how much" -- it comes from game theory (Shapley values, a fair way to split credit among contributors). It's mandatory because a security analyst will never act on an alert they can't understand or justify -- a black-box "trust me" score is not usable in a real SOC (Security Operations Centre).

Q. How do you know the model isn't cheating -- seeing the answer during training?

A. Time-aware, held-out splits: training and test data come from completely separate captures/campaigns, so the model never sees any part of a test scenario during training. Alert thresholds are also chosen only on a separate validation set and then applied unchanged to the test set -- never tuned on the data being reported.

Q. Why does the model add an "Impact" stage the problem statement didn't name?

A. Denial-of-service floods and ransomware encryption are real, common attacks that don't fit cleanly into the five named stages (Recon/Initial Access/Lateral Movement/C2/Exfiltration) -- forcing a SYN flood to be labelled "Reconnaissance" just because both involve many packets would have corrupted what the model learns. Adding a sixth, honest category was the more correct modelling choice, and it's disclosed clearly rather than hidden.

Q. Your weakest stage only scores 0.50 -- why?

A. The "Impact" stage only had 32 examples in the test set -- not enough data to learn it well. Every other stage scores 0.78-0.90. This is reported openly in the benchmark document rather than cherry-picked away.

C. Backend / engineering questions

Q. What inputs does it accept?

A. A raw PCAP/PCAPNG packet capture, or a flow-level CSV in the format tools like CICFlowMeter produce (it auto-detects CIC-IDS2018/2017, UNSW-NB15 and CTU-13 CSV formats) -- exactly the input the problem statement specifies.

Q. Do you use scapy or Wireshark to read the PCAP file?

A. No -- the PCAP/PCAPNG parser is written from scratch for this project. That was a deliberate choice: fewer third-party dependencies means fewer things that can break or be exploited, which matters on a network that's supposed to stay air-gapped from the internet.

Q. Does it need an internet connection to run?

A. No. Everything -- parsing, feature extraction, the AI model, the explanation, the dashboard -- runs entirely on the local machine. No cloud API calls, no telemetry, nothing is ever uploaded anywhere.

Q. How fast is it? Would it scale?

A. Under 4 seconds for an ~89,000-packet, ~4,700-flow capture on an ordinary laptop with no GPU. For production-scale, continuous traffic this prototype would need to be paired with a proper packet-capture pipeline (a network TAP/SPAN port feeding it continuously) -- which is an engineering integration step, not a change to the model itself.

Q. Where does your training data come from -- is it real traffic?

A. Two sources: a built-in synthetic generator that creates labelled traffic with a real, timestamped kill-chain (so the ground truth is known exactly, which real-world captures rarely give you), and adapters for standard public research datasets (CIC-IDS2018, CIC-IDS2017, UNSW-NB15, CTU-13) that are the accepted benchmarks in this field. The synthetic data is clearly labelled as synthetic everywhere it's used, not presented as real-world traffic.

Q. How do you test that the code actually works?

A. 242 automated tests, all written using Python's own built-in testing tools (no extra install needed), so they run anywhere, including offline. They cover the parser, the feature math, the windowing logic, the model, and the explainability code.

D. Frontend / dashboard questions

Q. Walk me through what happens when I use the dashboard.

A. You open a local web page, drag in a PCAP or CSV file, optionally adjust the alert threshold and how many steps ahead to forecast, and click Forecast. Within a few seconds you see the peak risk score, the predicted attack stage, a risk-over-time chart with the forecast shown as a shaded region, a breakdown of which features drove the prediction, and the actual network flows behind the alert.

Q. Is the dashboard connected to real code, or is it a mockup with fake numbers?

A. It's real -- the same analysis engine the command line uses. This was verified today by uploading an actual capture through the dashboard and confirming the numbers matched a fresh run of the underlying engine.

Q. Can this be used without the web dashboard?

A. Yes -- there's a command-line tool that does the exact same analysis and prints a full report to the terminal, useful for scripting or for an analyst who prefers a terminal.

E. Deployment, security & "gotcha" questions

Q. Is this ready for NTRO to deploy in production today?

A. Honestly: this is a hackathon prototype that proves the core idea -- world-model forecasting with MITRE mapping and explainability -- works end-to-end on real traffic, in under 4 seconds, fully offline. Production deployment would additionally need live integration with a network tap/SPAN port, validation on much larger real-world operational traffic, and hardening -- that's the honest next step, not something to claim is already done.

Q. What if an attacker knows about your model and tries to evade it?

A. That's an open research question for every ML-based detector, not unique to this project. Two practical mitigations are already built in: the independent rule-engine cross-check (so a purely learned-model blind spot doesn't go unnoticed), and explainability itself, so a human analyst stays in the loop rather than trusting an opaque score.

Q. Why not just use an existing tool like Splunk, Darktrace, or a commercial EDR?

A. Those are strong products, but they're either signature/anomaly-based (not built to forecast a kill-chain forward in time with an explicit MITRE stage output), proprietary and cloud-connected (a problem for an air-gapped network), or both. The problem statement specifically asks for a forecasting world model with explainability -- that's a narrower, more research-forward ask than what off-the-shelf SOC tools provide out of the box.

Q. Does it store or send any of my network data anywhere?

A. No. It runs entirely on the machine you start it on. Nothing is uploaded, logged externally, or sent to any cloud service.

Q. Did you actually test live packet capture, or is that just a claim?

A. It's real and was tested with real traffic -- but it's a bonus beyond what the problem statement asks for (file input), so it isn't the centrepiece of the demo. Happy to show it live if asked.

Part 7 -- Cheat Sheet (memorise these)

- F1 score: world model 0.92 vs 0.89 (context baseline) vs 0.84 (static/"traditional IDS" baseline)
- False-positive rate: 0.07 (world model) -- less than half of the static baseline's 0.17
- Stage classification accuracy: 84%
- State vector: 103 numbers describing each 30-second window of network activity
- Forecast horizon: K = 5 steps ahead (2.5 minutes), using 16 windows (8 minutes) of history
- Automated tests: 242, all pure standard-library Python -- no internet needed to run them
- Speed: under 4 seconds for an ~89,000-packet / ~4,700-flow capture
- Fully offline: no cloud APIs, no telemetry, nothing leaves the machine
- 6 MITRE ATT&CK stages modelled: Reconnaissance, Initial Access, Lateral Movement, Command & Control, Exfiltration, Impact
- Input formats: PCAP/PCAPNG, or flow CSV (CIC-IDS2018/2017, UNSW-NB15, CTU-13 auto-detected)

Part 8 -- Glossary (every term explained)

Term	Meaning
PCAP	"Packet CAPture" -- a file format that stores raw network traffic exactly as it travelled over the wire. Wireshark is the most famous tool for viewing these.
Flow	One "conversation" between two computers, identified by source IP/port, destination IP/port, and protocol -- e.g. one web page load is roughly one flow.
Feature	A measurable number describing traffic, used as an input to the AI model -- e.g. "number of distinct ports contacted" or "average packet size".
MITRE ATT&CK	A public, industry-standard catalogue of attacker behaviours and the stages an attack typically goes through (scanning, breaking in, spreading, stealing data, etc.), maintained by the MITRE Corporation and used across the cybersecurity industry as a shared vocabulary.
Kill chain	The step-by-step sequence an attacker follows to compromise a target, from first reconnaissance to final objective (e.g. stealing data).
World model	An AI model that learns how a system's internal state changes over time, so it can simulate/forecast the future rather than only classify the present.
K-step rollout / forward simulation	Running the model's own prediction back through itself K times in a row to forecast K steps into the future, the same way a weather model uses today's forecast to build tomorrow's.
Temporal transformer	A deep-learning architecture (from the same family that powers modern language models) that is good at finding patterns across a sequence of events over time.
Logistic regression	A simple, well-understood statistical model used here as the fair "traditional" baseline to compare the world model against.
Ridge regression	A simple linear statistical model with a mathematical safeguard against overfitting; used in ATDRPS's lightweight "linear-dynamics" backend.
SHAP (Shapley Additive exPlanations)	A method, based on game theory, for fairly splitting credit for a prediction among all the input features -- used here to explain *why* the model raised an alert.
False positive	A false alarm -- normal, harmless traffic that gets wrongly flagged as an attack. Too many of these and analysts start ignoring the tool.
F1 score	A single number (0 to 1) that balances catching real attacks against avoiding false alarms -- a standard way to score a detector fairly. Higher is better.
Air-gapped	A network that is physically or logically disconnected from the public internet, common in defence and critical-infrastructure settings for security.
IDS (Intrusion Detection System)	Any tool that watches network traffic and raises alerts on suspicious activity -- the general category ATDRPS belongs to, but with forecasting added.
Dashboard / frontend	The web page a human interacts with -- upload a file, see charts and results. It contains no logic of its own; it just displays what the backend computes.
Backend	The part of the program that does the actual computation -- parsing files, running the model -- with no visual interface of its own.
Flask	A small, widely-used Python library for building simple web servers -- what serves ATDRPS's dashboard locally.
CLI (Command-Line Interface)	Running the tool by typing commands in a terminal, instead of clicking in a web page -- useful for scripting or automation.
Offline / air-gap-safe deployment	Design choices (no cloud calls, no external dependencies that need the internet) that let a tool run entirely inside a disconnected network.

Generated from the ATDRPS project repository (SIH 2026, PS 26153) -- source: README.md, docs/ARCHITECTURE.md, docs/BENCHMARKS.md, docs/DEMO_SCRIPT.md, and the live-verified dashboard run on the demo capture, dated 15 September 2026.